

InternalMailExchangeService

On-premises Exchange mailbox client

EWS (on-premises)



Depends on mailbox location — read Section 1

notification-service · com.dbs.edoc.notification.services.notification.ews · Spring Boot / Java 17

1. IMPORTANT — Read Before Migrating This Class

This class may not need to change at all

The URL in this class — `webmail.uat1bank.dbs.com` — is an **on-premises Exchange Server** endpoint, not Exchange Online. Microsoft's EWS retirement announcement is explicit that it **applies only to Exchange Online / Microsoft 365**. On-premises Exchange Server is **not affected**: "there are no changes to EWS in Exchange Server" and "EWS will continue to be fully supported for Exchange on-premises mailboxes" (Microsoft Learn / Exchange Team blog, 2026). The 1 October 2026 blocking date that is driving `CloudMailExchangeService`'s migration simply does not apply to this class as it stands today.

So when would this class need to migrate?

Only if the mailboxes it talks to are moved off on-premises Exchange — e.g. as part of a broader plan to fully migrate to Exchange Online, or a Hybrid deployment. Two things worth knowing if that happens:

- Microsoft Graph **cannot talk to a true on-premises Exchange Server** directly — Graph is a Microsoft 365 cloud API. A mailbox has to actually live in (or be proxied through) Exchange Online / Exchange Hybrid for Graph calls to reach it.
- For Hybrid environments, Microsoft has stated that **Exchange SE** (the next on-premises Exchange release) is required to host on-premises mailboxes that need to be reachable via Graph — a plain older on-premises Exchange Server is not sufficient on its own.

Recommended next step

Before writing any migration code for this class, confirm with your infrastructure/Exchange admin team whether `webmail.uat1bank.dbs.com` mailboxes are (a) staying on-premises indefinitely — in which case this class can be left exactly as-is and simply revisited later, or (b) scheduled to move to Exchange Online/Hybrid — in which case Section 3 below shows the target-state code, which is functionally identical to `CloudMailExchangeService`'s migration once the mailboxes are cloud-hosted.

2. Current Implementation (On-premises EWS — unaffected by the Oct 2026 retirement)

InternalMailExchangeService.java – CURRENT (on-prem EWS)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.dbs.edoc.config.DynamicBooleanProperty;
import com.dbs.edoc.config.DynamicStringProperty;
import com.dbs.edoc.crypto.utils.ssl.IgnoreSSLCertificates;
import com.dbs.edoc.crypto.utils.ssl.InstallSSLCertificates;
import microsoft.exchange.webservices.data.core.ExchangeService;
import microsoft.exchange.webservices.data.core.enumeration.misc.ExchangeVersion;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.net.URI;
import java.util.Locale;

import static microsoft.exchange.webservices.data.core.enumeration.misc.ExchangeVersion.Exchange2010_SP2;

@Service
public class InternalMailExchangeService extends MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(InternalMailExchangeService.class);
    private static final DynamicBooleanProperty MAIL_EXCHANGE_SSL_IGNORE = new DynamicBooleanProperty("internal.exchange.mail.ssl.ignore",
false);
    private static final DynamicStringProperty EXCHANGE_SERVICE_URL = new DynamicStringProperty("internal.exchange.service.url",
"https://webmail.uatlbank.dbs.com/EWS/Exchange.asmx");
    private static final DynamicStringProperty EXCHANGE_SERVICE_HOST_PORT = new DynamicStringProperty("internal.exchange.service.host.port",
"webmail.uatlbank.dbs.com:443");
    private ExchangeService exchangeService;

    @Autowired
    public InternalMailExchangeService() {

        try {
            installMailExchangeCerts();
            this.exchangeService = new ExchangeService(Exchange2010_SP2);
            this.exchangeService.setUrl(new URI(EXCHANGE_SERVICE_URL.get()));
            LOGGER.info("Mail Exchange Service initialized to [{}]", EXCHANGE_SERVICE_URL.getValue());
        } catch (Exception e) {
            LOGGER.error("Exchange Server could not be initialized to [{}]. Will not be saving outgoing messages for on-premise email
accounts", EXCHANGE_SERVICE_URL.getValue());
        }
    }

    private void installMailExchangeCerts() {
        if (MAIL_EXCHANGE_SSL_IGNORE.getValue()) {
            IgnoreSSLCertificates.ignoreAll();
        } else {
            InstallSSLCertificates.installFor(EXCHANGE_SERVICE_HOST_PORT.getValue());
        }
    }

    @Override
    void initializeExchangeService(String username, String password, String domain) {
        // On-prem EWS: NTLM/basic auth against the internal Exchange server.
        // (credential wiring omitted from the captured screenshot)
    }

    @Override
    ExchangeService getExchangeService() {
        return this.exchangeService;
    }
}
```

What this class does today

It points an ExchangeService at the bank's internal on-premises Exchange host, installing (or bypassing, via MAIL_EXCHANGE_SSL_IGNORE) SSL certificates for that internal hostname, and hands that ExchangeService back to the shared saveMailToOutbox(...) logic in MailExchangeService. There is no Azure AD / OAuth2 involvement here at all — it's classic on-prem EWS, most likely authenticating via NTLM or Basic auth against the local domain.

3. Target-State Code — ONLY if/when mailboxes move to Exchange Online

InternalMailExchangeService.java – FUTURE STATE (Microsoft Graph, once mailboxes are cloud-hosted)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.azure.identity.ClientSecretCredential;
import com.azure.identity.ClientSecretCredentialBuilder;
import com.dbs.edoc.config.DynamicStringProperty;
import com.dbs.edoc.notification.error.ExchangeMessageException;
import com.dbs.edoc.notification.util.ProxyRegistry;
import com.microsoft.graph.serviceclient.GraphServiceClient;
import okhttp3.Credentials;
import okhttp3.OkHttpClient;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

import java.net.InetSocketAddress;
import java.net.Proxy;

// Applies ONLY once these mailboxes are hosted in Exchange Online (fully
// migrated to the cloud, or reachable via a Hybrid deployment running
// Exchange SE -- see the "Important" callout on page 1 before using this).
@Service
public class InternalMailExchangeService extends MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(InternalMailExchangeService.class);

    private GraphServiceClient graphServiceClient;

    private static final DynamicStringProperty CLIENT_ID = new DynamicStringProperty("internal.exchange.client.id", "");
    private static final DynamicStringProperty CLIENT_SECRET = new DynamicStringProperty("internal.exchange.client.secret", "");
    private static final DynamicStringProperty TENANT_ID = new DynamicStringProperty("internal.exchange.tenant.id", "");
    private static final String GRAPH_DEFAULT_SCOPE = "https://graph.microsoft.com/.default";

    public InternalMailExchangeService() {
        try {
            ProxyRegistry.authenticateProxy();

            OkHttpClient.Builder httpClientBuilder = new OkHttpClient.Builder();
            if (ProxyRegistry.getHost() != null && !ProxyRegistry.getHost().isEmpty()) {
                httpClientBuilder.proxy(new Proxy(Proxy.Type.HTTP,
                    new InetSocketAddress(ProxyRegistry.getHost(), Integer.parseInt(ProxyRegistry.getPort()))));
                httpClientBuilder.proxyAuthenticator((route, response) -> response.request().newBuilder()
                    .header("Proxy-Authorization",
                        Credentials.basic(ProxyRegistry.getUsername(), ProxyRegistry.getPassword()))
                    .build());
            }
            OkHttpClient httpClient = httpClientBuilder.build();

            ClientSecretCredential credential = new ClientSecretCredentialBuilder()
                .clientId(CLIENT_ID.getValue())
                .clientSecret(CLIENT_SECRET.getValue())
                .tenantId(TENANT_ID.getValue())
                .build();

            this.graphServiceClient = new GraphServiceClient(credential, httpClient, GRAPH_DEFAULT_SCOPE);

            LOGGER.info("Initialized Internal Mail Exchange Service (Microsoft Graph, app-only auth)");
        } catch (Exception ex) {
            LOGGER.error("Error occurred while initializing Internal Mail Exchange Service", ex);
        }
    }

    @Override
    void initializeExchangeService(String username, String password, String domain) throws ExchangeMessageException {
        // No-op for the same reason as CloudMailExchangeService: app-only
        // Graph auth needs no per-mailbox credentials.
        LOGGER.info("Application-only Graph auth in use; no per-user token needed for [{}]", username);
    }

    @Override
    GraphServiceClient getGraphServiceClient() {
        return graphServiceClient;
    }
}
```

4. What Would Change, If and When This Migrates

Concern	Before (on-prem EWS)	After (Graph, cloud-hosted mailboxes only)
Applicability of Oct 2026 retirement	Not affected — EWS stays fully supported on-premises	Only relevant once/if mailboxes are cloud-hosted
Client object	ExchangeService pointed at internal host URL	GraphServiceClient pointed at graph.microsoft.com (no internal URL configuration needed)
SSL certificate handling	InstallSSLCertificates / IgnoreSSLCertificates for the internal self-hosted certificate chain	Not needed — Graph's public endpoint uses a standard, publicly trusted certificate
Authentication	NTLM/Basic auth against the domain (per initializeExchangeService)	OAuth2 client-credentials flow via azure-identity, identical pattern to CloudMailExchangeService
Host/port configuration property	internal.exchange.service.host.port=webmail.uat1bank.dbs.com:443	Not needed

5. Configuration & Dependency Notes

If this class stays on-premises (most likely, for now)

No pom.xml or application.properties changes are required for this class. Keep `ews-java-api` as a dependency solely for this class if `CloudMailExchangeService` no longer needs it, and leave `internal.exchange.*` properties untouched.

If/when it migrates (mailboxes moved to Exchange Online / Hybrid with Exchange SE)

Apply the same pom.xml and application.properties changes documented for `CloudMailExchangeService` (`microsoft-graph`, `azure-identity`, Azure AD app registration with `Mail.Send` application permission), using an `internal.exchange.client.id` / `.client.secret` / `.tenant.id` property set (kept separate from the cloud properties in case different Azure AD app registrations or scoping are used for internal vs. externally-facing mailboxes).